

ARCHITECTURE DES DONNÉES — RAPPORT TECHNIQUE

MediaPulse

Plateforme d'Intelligence Médiatique pour l'Agrégation, le
Traitement et la Visualisation d'Articles de Presse

PROJET

MediaPulse

DATE

Mai 2026

AUTEUR

**Ayoub Lafdaigui et Safaa
Hassani**

VERSION

1.0 — Production

CODE SOURCE

**[github.com/ayoublafdaigui/
architecture-project](https://github.com/ayoublafdaigui/architecture-project)**

STACK

**Python · Kafka · MinIO ·
PostgreSQL**

Table des Matières

1. Contexte et Objectifs du Projet	3
1.1 Contexte médiatique marocain	3
1.2 Objectifs techniques	3
2. Architecture du Système	4
2.1 Vue d'ensemble	4
2.2 Couches de données (Medallion)	5
2.3 Flux de données de bout en bout	5
3. Choix Technologiques	6
3.1 Ingestion et streaming	6
3.2 Stockage des données	6
3.3 Pipeline de transformation	7
3.4 Orchestration et monitoring	7
4. Déploiement (Docker / Kubernetes)	8
4.1 Docker Compose	8
4.2 Inventaire des services	8
5. Documentation d'Installation et d'Utilisation	9
5.1 Prérequis	9
5.2 Démarrage rapide	9
5.3 CLI Opérateur	10
6. Dashboards de Visualisation	11
7. Démonstration Fonctionnelle	12
8. Récapitulatif des Livrables	13

1. Contexte et Objectifs du Projet

1.1 Contexte médiatique

Le paysage médiatique marocain est riche et fragmenté : des dizaines de sites d'information publient chaque jour des centaines d'articles en arabe, français et anglais. À côté de ces sources locales, les grandes chaînes internationales (Al Jazeera, BBC, CNN, Reuters) couvrent les mêmes événements avec un regard extérieur.

Il n'existait pas jusqu'ici d'outil unifié permettant de **collecter, normaliser, comparer et visualiser** cette information de manière automatisée et temps-réel.

Problème résolu : un analyste qui veut savoir « quels sujets dominent l'actualité marocaine cette semaine ? » devait visiter manuellement 8 sites, lire des articles en 3 langues et consolider ses notes dans Excel. MediaPulse automatise tout ce processus en quelques minutes.

1.2 Objectifs techniques



Objectif	Critère de succès
Collecte automatisée	Scraper 8 sources sans intervention humaine, toutes les X heures via Airflow
Streaming fiable	Articles transmis via Kafka avec garantie de livraison, consumer écrit en Bronze MinIO
Qualité des données	Pipeline Bronze→Silver avec 6 règles de validation, rejet traçable des enregistrements invalides
Agrégation analytique	Couche Gold avec top-keywords, tendances quotidiennes, répartition géographique
Entrepôt de données	Schéma en étoile PostgreSQL (dim_source, dim_date, fact_articles, fact_keyword_frequency)

Objectif	Critère de succès
Visualisation	Dashboards Grafana provisionnés automatiquement ; métriques Prometheus en temps réel

Sources couvertes

Source	Pays	Langue	Type
Hespress	MA Maroc	Arabe	Nationale
Akhbarona	MA Maroc	Arabe	Nationale
Barlamane	MA Maroc	Arabe	Nationale
Lakom (Lakome2)	MA Maroc	Arabe	Nationale
Al Jazeera	QA Qatar	Anglais	Internationale
BBC News	GB Royaume-Uni	Anglais	Internationale
CNN	us États-Unis	Anglais	Internationale
Reuters	GB Royaume-Uni	Anglais	Internationale

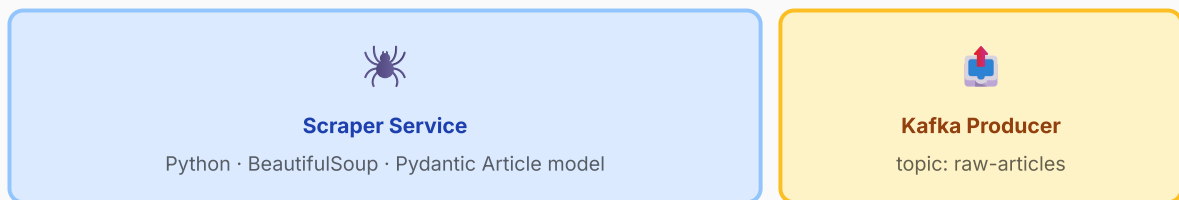
2. Architecture du Système

2.1 Vue d'ensemble — Schéma d'architecture

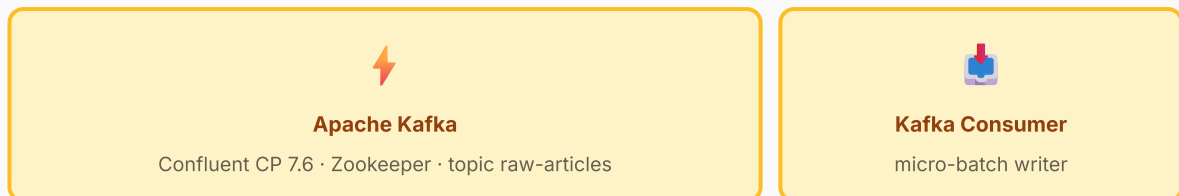
SOURCES EXTERNES



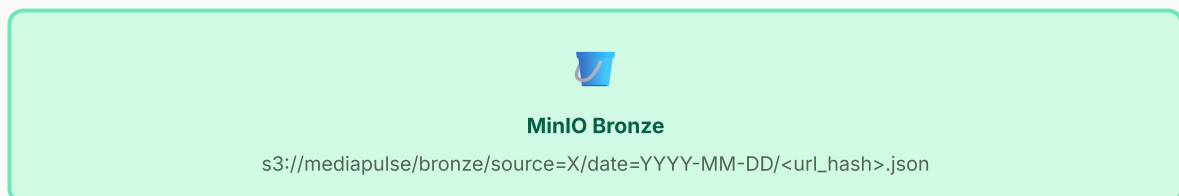
COUCHE D'INGESTION



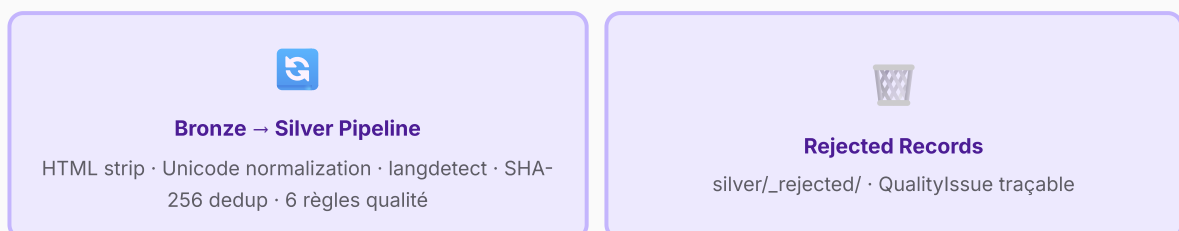
COUCHE STREAMING

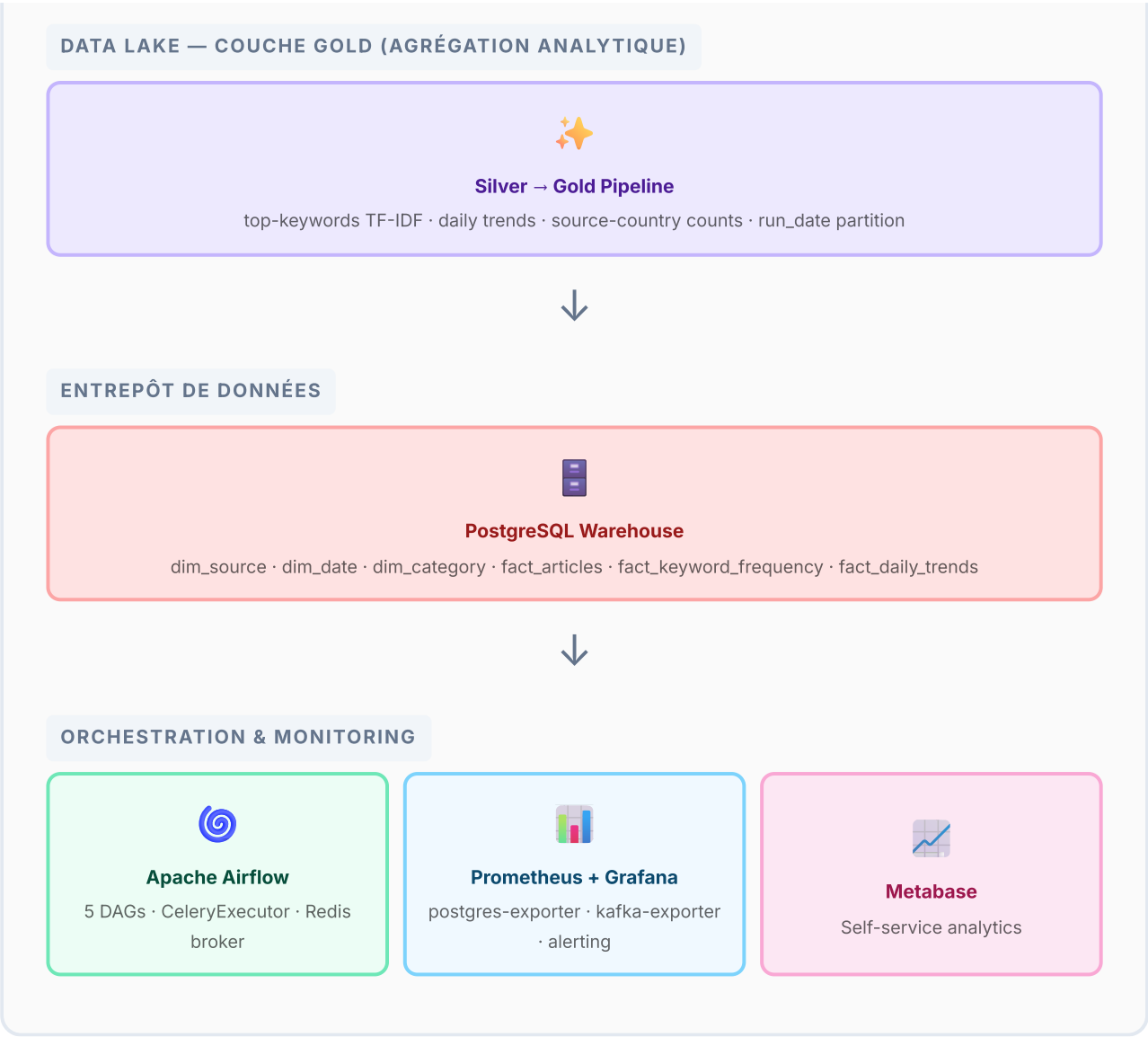


DATA LAKE — COUCHE BRONZE



DATA LAKE — COUCHE SILVER (NETTOYAGE & VALIDATION)





2.2 Architecture Medallion — Trois couches de données

Couche	Chemin MinIO	Format	Rôle
 Bronze	bronze/source=X/date=YYYY-MM-DD/<hash>.json	JSON brut	Donnée telle que scrapée, immuable. Source de vérité.
 Silver	silver/source=X/date=YYYY-MM-DD/<hash>.json	JSON nettoyé	HTML strippé, encodage normalisé, langue détectée, déduplicué.
 Gold	gold/table=X/run_date=YYYY-MM-DD/data.json	JSON agrégé	Top keywords (TF-IDF), tendances, comptages journaliers par source.

2.3 Flux de données de bout en bout

1 Scraping (batch ou streaming)

Le `GenericArticleScraper` crawle les pages d'index de chaque source, identifie les URLs d'articles via regex, télécharge et parse le HTML avec BeautifulSoup. Chaque article est validé par le modèle Pydantic `Article` (titre, contenu, URL, langue, pays, timestamps).

2 Publication Kafka

Le `KafkaProducer` sérialise l'article en JSON et le publie sur le topic `raw-articles`. En mode batch direct, le `BronzeWriter` écrit directement dans MinIO sans passer par Kafka.

3 Consommation et stockage Bronze

Le `KafkaConsumer` lit les messages par micro-lots de 50, les persiste sous `bronze/source=X/date=D/<url_hash>.json` dans MinIO. Le chemin est partitionné par source et par date de publication.

4 Transformation Bronze → Silver

Le pipeline nettoie les champs (strip HTML, NFKC Unicode, entités HTML), détecte la langue via `langdetect`, calcule un hash SHA-256 de l'URL pour la déduplication. Les records qui échouent à au moins une des 6 règles de validation sont rejetés vers `silver/_rejected/` avec la raison de rejet.

5 Agrégation Silver → Gold

Lecture des fichiers Silver, tokenisation multi-langue (arabe + latin), calcul TF-IDF pour les top-50 keywords, agrégation des tendances quotidiennes par sujet, comptages par source et par pays. Résultats écrits en Gold partitionné par `run_date`.

6 Chargement Warehouse

Le `WarehouseLoader` lit les fichiers Gold, résout les clés de dimension (upsert dans `dim_source`, `dim_date`, `dim_category`) puis insère les faits dans `fact_articles`, `fact_keyword_frequency`, `fact_daily_trends`.

7 Visualisation

Grafana se connecte au warehouse PostgreSQL via le datasource provisionné automatiquement. Le dashboard *MediaPulse News Intelligence* affiche les comptes journaliers, la répartition linguistique, les top-keywords et le score de qualité en temps réel.

3. Choix Technologiques et Justifications

3.1 Ingestion et Streaming

Python 3.11

LANGAGE PRINCIPAL

Riche écosystème web scraping (requests, BeautifulSoup, lxml), NLP (spaCy, langdetect), data (pandas). Typage strict avec Pydantic.

Apache Kafka (Confluent 7.6)

BUS DE MESSAGES

Découplage producteur/consommateur, durabilité des messages, possibilité de replay. Indispensable pour un pipeline de scraping à haute fréquence. Topic `raw-articles` avec réplication factor 1 (dev).

Pydantic v2

VALIDATION DES DONNÉES

Modèle `Article` avec validation stricte (URL, langues ISO 639-1, timestamps UTC). Rejet immédiat des données malformées dès l'ingestion.

BeautifulSoup4 + lxml

PARSING HTML

Parsing robuste des pages d'actualité. Sélection d'articles par patterns regex d'URL. Tolérant aux HTML malformés courants sur les sites d'information.

3.2 Stockage des Données

MinIO

DATA LAKE (BRONZE/SILVER/GOLD)

Compatible API S3, déployable on-premise. Stockage objet partitionné par `source` et `date` — permet un scan sélectif sans lire l'intégralité du lac. Remplace S3 AWS dans un contexte académique.

PostgreSQL 16

ENTREPÔT ANALYTIQUE

Schéma en étoile sous le namespace `warehouse`. Extensions `pgcrypto` pour les UUIDs. Vues SQL pré-calculées pour les dashboards (`daily_article_counts`, `language_distribution`, `data_quality_score`).

Redis

BROKER CELERY (AIRFLOW)

Broker léger pour les tasks Airflow CeleryExecutor. Faible latence, simple à opérer en développement.

Architecture Medallion

DESIGN PATTERN

Bronze (raw), Silver (curé), Gold (agrégé) — chaque couche est immuable et rejouable. Permet un debugging progressif et une traçabilité complète de la data lineage.

3.3 Pipeline de Transformation

langdetect

DÉTECTION DE LANGUE

Détection automatique de l'arabe, anglais, français. Seed fixe (=0) pour la reproductibilité. Fallback sur le `language_hint` de la source en cas d'échec.

spaCy

NLP / TOKENISATION

Tokenisation multilingue pour l'extraction de keywords. Support du bloc Unicode arabe (U+0600-U+06FF). Calcul TF-IDF fait maison sur base de la fréquence brute.

pandas

AGRÉGATION GOLD

Lecture et agrégation des fichiers Silver JSON en DataFrames. Calcul des comptages journaliers, top-keywords, tendances par source.

python-dateutil

NORMALISATION TEMPORELLE

Parsing de dates dans tous formats (ISO, RFC, timestamps journaux). Conversion systématique en UTC pour la cohérence dans le warehouse.

3.4 Orchestration, Visualisation et Monitoring

Apache Airflow 2.9.3

ORCHESTRATION

5 DAGs : scrape batch, Bronze→Silver, Silver→Gold, chargement warehouse, contrôle qualité. CeleryExecutor pour la scalabilité horizontale. DAGs versionnés avec le code source.

Grafana 11.1

DASHBOARDS

Provisionnement automatique (datasource PostgreSQL + Prometheus). Dashboard *MediaPulse News Intelligence* défini en JSON, versionnée dans le repo.

Prometheus

MÉTRIQUES SYSTÈME

Scrape le postgres-exporter (port 9187) et le kafka-exporter (port 9308). Règles d'alerte pour données périmées et chutes du score de qualité.

Metabase

ANALYTICS SELF-SERVICE

Interface no-code pour explorer le warehouse PostgreSQL. Idéal pour les équipes métier non-techniques. Connecté à la même base que Grafana.

4. Déploiement

4.1 Docker Compose

L'intégralité du stack est containerisé. Le fichier `mediapulse/docker-compose.yml` définit tous les services, réseaux et volumes. Les images applicatives (scraper, Airflow) sont construites localement à partir de `Dockerfile` (Python 3.11-slim) et `Dockerfile.airflow`.

```
# Démarrage complet du stack
Copy-Item mediapulse\.env.example mediapulse\.env # configurer les secrets
cd mediapulse
docker compose up --build

# Mode demo (seed warehouse + Grafana prêt)
docker compose --profile demo up --build demo-seed
```

4.2 Inventaire des services

Service	Image	Port exposé	Rôle
<code>zookeeper</code>	confluentinc/cp-zookeeper:7.6.1	2181	Coordination Kafka
<code>kafka</code>	confluentinc/cp-kafka:7.6.1	9092	Bus de messages
<code>minio</code>	minio/minio (2024-07)	9000, 9001	Data Lake S3-compatible
<code>minio-init</code>	minio/mc	—	Création du bucket au démarrage
<code>postgres</code>	postgres:16	5432	Warehouse analytique
<code>redis</code>	redis:7-alpine	6379	Broker Celery Airflow
<code>airflow-init</code>	mediapulse-airflow:latest	—	Migration DB + création admin
<code>airflow-webserver</code>	mediapulse-airflow:latest	8080	UI Airflow

Service	Image	Port exposé	Rôle
<code>airflow-scheduler</code>	mediapulse-airflow:latest	—	Planification des DAGs
<code>airflow-worker</code>	mediapulse-airflow:latest	—	Exécution des tasks Celery
<code>scraper</code>	Dockerfile local	—	Service de scraping batch
<code>stream-consumer</code>	Dockerfile local	—	Consumer Kafka → Bronze
<code>warehouse-migrate</code>	Dockerfile local	—	Application du schéma SQL
<code>metabase</code>	metabase/metabase:v0.50.18	3000	Analytics self-service
<code>prometheus</code>	prom/prometheus:v2.53.1	9090	Collecte de métriques
<code>postgres-exporter</code>	prometheuscommunity/postgres-exporter	9187	Métriques PostgreSQL
<code>kafka-exporter</code>	danielqsj/kafka-exporter	9308	Métriques Kafka
<code>grafana</code>	grafana/grafana:11.1.0	3001	Dashboards de visualisation

Réseau et volumes

Tous les services partagent le réseau bridge `mediapulse-net`. Les volumes nommés (`kafka-data`, `minio-data`, `postgres-data`, `grafana-data`, etc.) assurent la persistance des données entre les redémarrages.

5. Documentation d'Installation et d'Utilisation

5.1 Prérequis

Outil	Version minimale	Usage
Docker Desktop	24+	Containerisation de tous les services
Docker Compose	v2.20+	Orchestration multi-services
Python	3.11+	Exécution CLI locale (optionnel)
Git	2.40+	Clonage du dépôt
RAM disponible	8 Go minimum	Stack complet (Kafka + Airflow + Postgres)

5.2 Démarrage rapide (5 minutes)

Étape 1 — Cloner le dépôt

```
git clone https://github.com/ayoublafdaigui/architecture-project.git
cd architecture-project
```

Étape 2 — Configurer l'environnement

```
Copy-Item mediapulse\.env.example mediapulse\.env
# Éditer mediapulse\.env pour définir les mots de passe
```

Important : Ne pas committer le fichier `.env` — il est exclu par `.gitignore` . Les valeurs par défaut fonctionnent pour le développement local.

Étape 3 — Démarrer le stack

```
cd mediapulse
docker compose up --build
```

Première exécution : ~5-10 min (téléchargement des images). Les démarrages suivants : ~60 secondes.

Étape 4 — Vérifier les services

Service	URL	Identifiants par défaut
MinIO Console	http://localhost:9001	<code>mediapulse_minio</code> / <code>.env</code>
Airflow	http://localhost:8080	<code>admin</code> / <code>admin</code>
Metabase	http://localhost:3000	Wizard à la première connexion
Prometheus	http://localhost:9090	—
Grafana	http://localhost:3001	<code>admin</code> / <code>.env GF_*</code>
PostgreSQL	localhost:5432	<code>mediapulse</code> / <code>.env</code>

5.3 CLI Opérateur

Depuis le répertoire parent `architecture-project/`, le CLI `python -m mediapulse` offre toutes les commandes du pipeline :

```
# Diagnostic de connectivité (MinIO, Kafka, PostgreSQL)
python -m mediapulse doctor

# Appliquer le schéma warehouse (DDL)
python -m mediapulse apply-schema

# Scraper toutes les sources (batch)
python -m mediapulse scrape

# Scraper une source spécifique
python -m mediapulse scrape --source hespress

# Pipeline Bronze → Silver (toutes sources)
python -m mediapulse bronze-to-silver

# Pipeline Silver → Gold
python -m mediapulse silver-to-gold

# Charger le warehouse depuis Gold
python -m mediapulse load-warehouse

# Rapport de qualité
python -m mediapulse quality --layer silver
python -m mediapulse quality --layer gold

# Seed de données de démonstration (14 jours)
python -m mediapulse seed-demo --days 14 --articles-per-day 16
```

Exécution des tests

```
cd architecture-project
python -m pytest mediapulse/tests/ -v
```

Fichier de test	Ce qui est testé
test_article_model.py	Validation Pydantic Article (title, URL, langue, timezone)
test_bronze_to_silver.py	Pipeline de nettoyage, règles qualité, déduplication URL
test_silver_to_gold.py	Agrégation TF-IDF, tendances, comptages Gold
test_demo_seed.py	Génération de données de démonstration cohérentes

6. Dashboards de Visualisation

6.1 Grafana — MediaPulse News Intelligence

Le dashboard principal est **provisionné automatiquement** au démarrage de Grafana. Il est défini dans `mediapulse/dashboard/grafana/provisioning/dashboards/mediapulse-overview.json` et versionné dans Git — aucune configuration manuelle nécessaire.

Panel	Source de données	Indicateur
Articles par jour	Vue <code>warehouse.daily_article_counts</code>	Évolution temporelle du volume d'articles par source
Répartition linguistique	Vue <code>warehouse.language_distribution</code>	Proportion arabe / anglais / autres
Top Keywords	<code>warehouse.fact_keyword_frequency</code>	Mots-clés les plus fréquents du jour (TF-IDF)
Tendances du jour	<code>warehouse.fact_daily_trends</code>	Sujets trending avec score et comptage
Score de qualité	Vue <code>warehouse.data_quality_score</code>	% de records passant les règles de validation par couche
Métriques Kafka	Prometheus kafka-exporter	Consumer lag, messages produits/consommés par minute
Métriques PostgreSQL	Prometheus postgres-exporter	Connexions actives, taille des tables, latence des requêtes

6.2 Alertes Prometheus

Les règles d'alerte définies dans `dashboard/prometheus-alerts.yml` couvrent :

- **Données périmées** : aucun nouvel article depuis > 6 heures → alerte `StaleData`
- **Score qualité bas** : taux de rejet Silver > 20% → alerte `HighRejectionRate`
- **Consumer lag Kafka** : lag > 1000 messages → alerte `KafkaConsumerLag`
- **PostgreSQL indisponible** : scrape echoue → alerte `PostgresDown`

6.3 Metabase — Exploration self-service

Metabase (port 3000) se connecte à la même base PostgreSQL. Les équipes non-techniques peuvent créer leurs propres questions et tableaux de bord sans écrire de SQL, en pointant directement sur les tables du schéma `warehouse`.

Accès rapide aux dashboards : après `docker compose up`, ouvrir Grafana sur **`http://localhost:3001`** → dossier *MediaPulse* → *MediaPulse News Intelligence*. Les données de démonstration sont disponibles via `docker compose --profile demo up demo-seed`.

7. Démonstration Fonctionnelle du Pipeline de Bout en Bout

7.1 Scénario de démonstration

La démonstration ci-dessous montre le pipeline complet depuis le scraping jusqu'à la visualisation, en environ 15 minutes avec les données de démonstration pré-générées.

A

Démarrer le stack avec données de démonstration

```
docker compose --profile demo up --build demo-seed
```

Démarre tous les services et génère 14 jours × 16 articles par source dans le warehouse PostgreSQL.

B

Observer le scraping en temps réel

Ouvrir Airflow sur <http://localhost:8080> → activer le DAG `mediapulse_batch_scrape` → déclencher manuellement. Suivre les logs en temps réel : scraping Hespress, Akhbarona, BBC, etc.

C

Vérifier le Bronze dans MinIO

Ouvrir <http://localhost:9001> → bucket `mediapulse` → naviguer dans `bronze/` pour voir les articles JSON partitionnés par `source=X/date=YYYY-MM-DD/`.

D

Déclencher le pipeline Bronze → Silver → Gold

Dans Airflow, déclencher successivement `mediapulse_bronze_to_silver` puis `mediapulse_silver_to_gold`. Observer le décompte d'articles acceptés vs rejetés dans les logs.

E

Charger le warehouse et voir les dashboards

Déclencher `mediapulse_load_warehouse`. Puis ouvrir Grafana (<http://localhost:3001>) → dashboard *MediaPulse News Intelligence* : les graphiques se peuplent avec les données réelles scrapées.

7.2 Règles de validation Silver démontrées

Règle	Condition de rejet
<code>title_not_empty</code>	Titre absent ou vide après nettoyage
<code>published_at_not_null</code>	Date de publication manquante ou non parseable
<code>content_length_gt_100</code>	Contenu $\leq$ 100 caractères (publicité, nav menu)
<code>url_valid</code>	URL sans scheme http/https ou sans domaine
<code>url_hash_not_empty</code>	Hash SHA-256 de l'URL manquant
<code>url_unique</code>	URL déjà vue dans la même exécution (doublon)

7.3 Modèle de données — Entrepôt en étoile

dim_source	dim_date	fact_articles (centre)	fact_keyword_frequency	fact_daily_trends
source_key (PK) source_name country_code source_type language_hint	date_key (PK) full_date year/month/day day_name is_weekend	article_key (PK) ur_hash (UNIQUE) source_key (FK) published_date_key (FK) title, content_text language_code sentiment_score topic_tags	date_key (FK) source_key (FK) keyword frequency_count tf_idf_score rank_position	date_key (FK) source_key (FK) topic article_count trend_score top_keywords[]

8. Récapitulatif des Livrables

✓ Présentation détaillée du projet

Ce document couvre le contexte, l'architecture, les choix technologiques et la démonstration fonctionnelle.

✓ Schéma d'architecture

Diagramme complet en Section 2 : couches d'ingestion, streaming, data lake (Bronze/Silver/Gold), warehouse et visualisation.

✓ Code source versionné sur Git

Dépôt : <https://github.com/ayoublafdaigui/architecture-project>

59 fichiers — scrapers, modèles, pipelines, DAGs Airflow, tests, SQL, config Docker.

✓ Fichiers de déploiement Docker

`mediapulse/docker-compose.yml` · `mediapulse/Dockerfile` ·
`mediapulse/Dockerfile.airflow` — 18 services containerisés.

✓ Documentation technique d'installation et d'utilisation

Section 5 de ce document + `mediapulse/README.md` avec toutes les commandes CLI, URLs de services et instructions étape par étape.

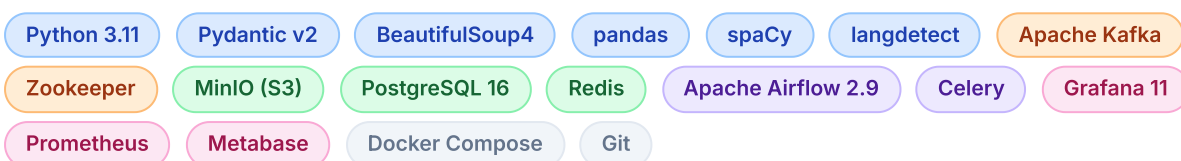
✓ Dashboards de visualisation

Dashboard Grafana provisionné automatiquement (`mediapulse-overview.json`). Alertes Prometheus. Interface Metabase pour analytics self-service.

✓ Démonstration fonctionnelle du pipeline de bout en bout

Section 7 : scénario A→E depuis le scraping jusqu'aux dashboards. Mode demo seed disponible via `docker compose --profile demo up demo-seed`.

Stack technologique — Récapitulatif



Dépôt GitHub : <https://github.com/ayoublafdaigui/architecture-project>

Branche principale : `main` — 59 fichiers, 5 812 lignes de code